Design and Implementation of a Cone-Guided Navigation System Using ROS 2 and TurtleBot4

Group 04: Agostino Cardamone [2276], **Chiara Ferraioli** [2169], **Asja Antonucci** [2437]

[1]Dept. of Information Eng., Electrical Eng. and Applied Mathematics - University of Salerno, Italy

Abstract This project explores the design of a ROS 2-based navigation system for a TurtleBot4 robot tasked with autonomously reaching a target location from a known starting point within a structured environment. The aim is to develop an architecture capable of computing the shortest viable path while avoiding obstacles and complying with directional constraints defined by the presence of color-coded cones. Along the route, the robot is required to pass consistently to the left or right of each cone depending on its color, with time penalties applied for any rule violations or for emergency interventions requiring manual repositioning. To ensure that testing remains conclusive, each trial is subject to a maximum time limit. Among the design objectives is also the ability to address the kidnapped robot problem, requiring the system to recover from sudden displacements. The proposed architecture integrates perception, planning, and control within a modular ROS 2 framework, with the goal of supporting robust and rule-aware autonomous navigation in evaluation scenarios defined by strict operational constraints.

For correspondence:

a.cardamone7@studenti.unisa.it (AC); c.ferraioli30@studenti.unisa.it (CF); a.antonucci5@studenti.unisa.it (AA)

1 Introduction

As mobile robotic systems become increasingly relevant in structured environments, their ability to navigate autonomously while complying with task-specific constraints is critical for deployment in real-world scenarios. This project focuses on the development of a ROS 2-based navigation architecture for the TurtleBot4 platform, designed to operate within a known environment and follow a predefined mission under strict rules and time constraints.

The robot is tasked with reaching a specified goal from a known starting point by computing the shortest feasible path through a mapped indoor space. Along the way, it must avoid both static and dynamic obstacles while obeying specific traversal rules imposed by the presence of color-coded cones. Depending on the color, each cone must be passed on either the left or right side, with time penalties applied for incorrect passages or for any emergency intervention requiring manual repositioning of the robot. To ensure consistent and bounded evaluations, a fixed time limit is enforced for each test run.

The TurtleBot4 is equipped with a variety of onboard sensors that enable perception and interaction with the environment. Among these, the RGB and stereo cameras provide image and depth data streams that support

scene understanding and spatial analysis. These visual sensors are particularly important for detecting and interpreting the color-coded cones, which are too small to be reliably detected by the robot's LIDAR sensor. The LIDAR, however, is essential in obstacle detection and collision avoidance, offering a reliable scan of the surrounding area to support safe navigation.

Since the robot operates within a known environment, it can exploit prior map information for global path planning and localization. Nevertheless, challenges such as the kidnapped robot problem, where the robot is displaced unexpectedly and must re-establish its position, require the system to be resilient to loss of localization and capable of reorientation using sensory input.

The system described in this report has been developed with the goal of meeting the specific requirements defined by the project, focusing on reliable navigation, correct rule handling, and effective use of sensor data within a known environment. The following sections describe the design choices, implementation steps, and technical components that support the robot's ability to carry out the assigned task under the given constraints.

2 ROS 2 Based Architecture

The architecture is designed to be modular and reactive, reflecting the need for real-time perception, decision-making, and motion control required by the task. ROS 2's **publish-subscribe communication** model is used throughout the system, allowing individual components to operate independently while exchanging data efficiently through topics.

This choice supports parallel processing of sensory input, state estimation, and actuation commands, which is essential for ensuring that the robot can detect cones, avoid both static and dynamic obstacles, and update its position continuously during navigation. While some of the system's external dependencies, such as the **navigation stack**, make internal use of services or action servers, all custom components developed for this project communicate exclusively through topics.

An alternative implementation strategy would have been to design and develop a fully custom behavioral tree to control the robot's execution flow. This approach would have allowed finer-grained control over task sequencing, failure recovery, and reactive behaviors, capabilities that could better accommodate the specific constraints of the challenge. However, this option was ultimately not pursued due to a combination of factors. Although the theoretical foundations of behavioral trees had been addressed in the course, we lacked hands-on experience with their implementation and integration in a ROS 2 context. Developing a complete behavioral tree from scratch would have required additional time to bridge that gap, understand the relevant APIs, and validate the behavior under realistic conditions. By contrast, the development of a modular architecture using standard ROS 2 components was a more straightforward path, largely due to our prior experience with ROS 1. This background allowed us to quickly adapt to the ROS 2 environment and focus our efforts on designing, implementing, and testing the core components of the system within the available time.

The remainder of this section presents the nodes and modules that compose the system, describing how they interact to handle perception, path execution, and compliance with the rules defined in the task.

2.1 Cone Detection Node

As the name suggests, the `cone_detection_node` is responsible for identifying cones within the scene observed by the robot. To accomplish this, it subscribes to the RGB and stereo camera topics to receive, respectively, color frames of the environment and a depth image containing per-pixel distance information. In this context, the stereo camera is essential: although the robot is equipped with a LIDAR sensor, the cones used in this scenario are not tall enough to intersect the laser scan plane, making them effectively invisible to that sensor. For this reason, the depth image provided by the stereo camera

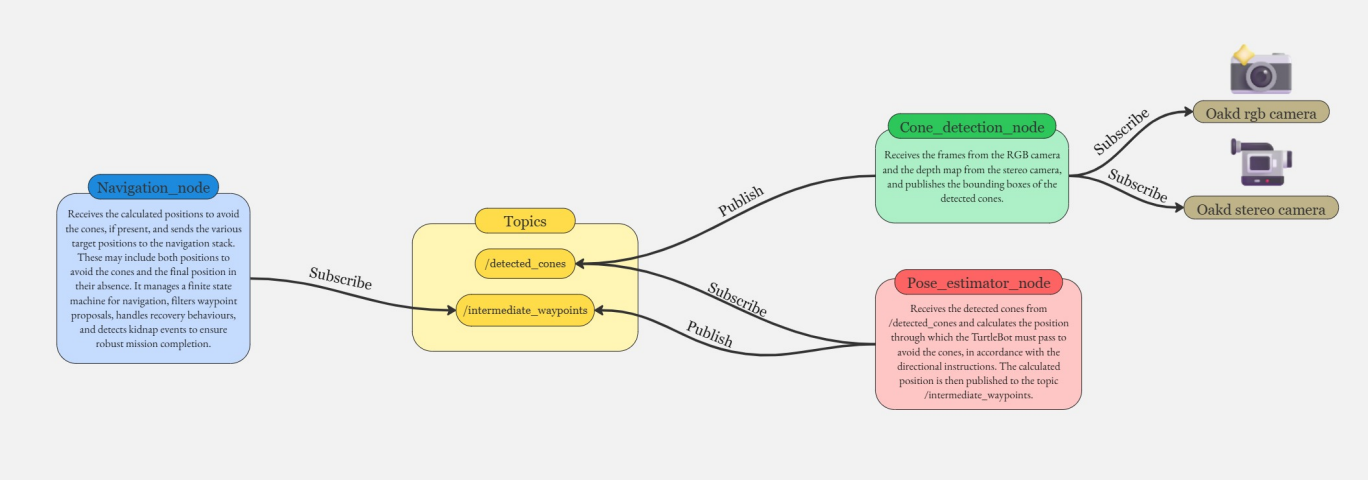

Figure 1: Design of the ROS 2-based Architecture

is used as the primary source for estimating cone distances throughout the detection process.

2.1.1 Detection Pipeline

Each frame received from the RGB camera is first converted into a format compatible with **OpenCV**, enabling further image processing operations. A **YOLOv8**-based object detection model, obtained from the public repository by EngrAwab [1], is then applied to the frame to identify potential cones. The output is a list of bounding boxes corresponding to the detected objects.

However, due to the limitations of the detection model and the visual complexity of the test environment, a number of false positives are typically observed, particularly involving red or yellow-colored elements such as doors, signs, fire extinguishers, or floor stickers. To address this, a multi-stage filtering process is applied.

First, detections appearing near the upper and lower boundaries of the frame are systematically excluded. These regions are more prone to false positives, since background structures or unrelated objects often fall within them and are erroneously classified as cones.

In addition, cones captured in these peripheral bands are typically either too far away or too close, which makes depth estimation unstable. For this reason, a narrow mask is applied at the top and bottom of each frame, effectively suppressing false positives while preserving valid detections in the central area of the image.

Remaining detections are then filtered based on the size and aspect ratio of their bounding boxes. Extremely small detections or those with unusual height-to-width ratios are rejected, as these are often caused by reflections or lighting artefacts rather than actual cones.

Despite these refinements, an additional critical limitation was observed in scenes with extreme lighting conditions. In particular, cones placed in areas of intense illumination, such as corridors where sunlight entered and reflected directly off the floor, were often completely ignored by the detector. Attempts to adjust camera parameters like exposure, or to pre-process the image before detection, did not resolve the issue. This made the system vulnerable to failure in highly reflective or overexposed environments, where the cones effectively became invisible to the model.

Once a set of valid bounding boxes has been obtained, the system performs color classification to distinguish cones by type. This is necessary because the task specifies that the robot must pass either to the left or the right of a cone, depending on its color.

An initial approach based solely on HSV thresholding proved to be unreliable due to the highly variable lighting conditions in the testing environment. To improve

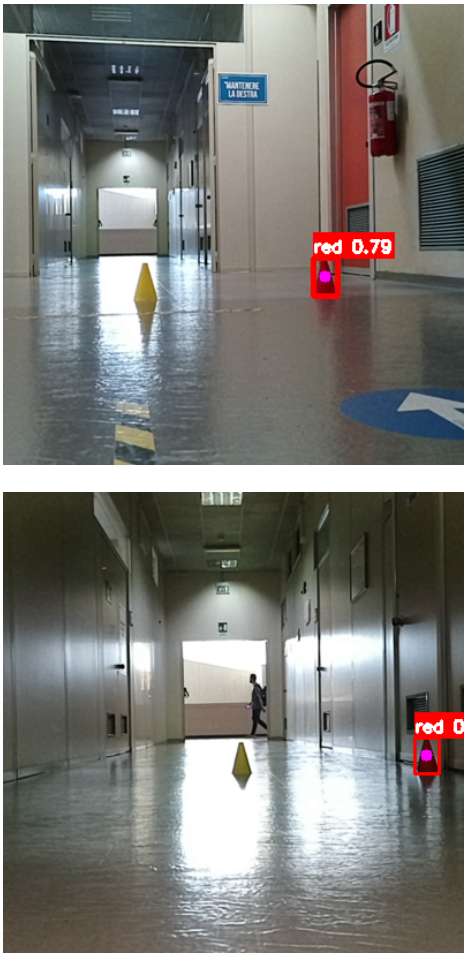

Figure 2: Cone Detection in Extreme Light Conditions

robustness, a multi-stage image processing pipeline was introduced, inspired by the methodology of Lin et al. [2], and adapted to the specific requirements of our scenario. The goal of this pipeline is to emphasize the relevant features of the cones and suppress background interference.

The processing pipeline includes:

- **Adaptive Color Boosting**: dynamically enhances saturation and brightness based on overall lighting conditions, making colors more distinguishable in both dark and bright scenes.
- **Background Suppression**: selectively reduces the visual influence of typical background elements such as white walls or brown surfaces, using heuristic HSV thresholds.
- **Target Color Enhancement**: increases saturation and brightness specifically for hues associated with cones (e.g., red, yellow, orange), improving separation from the background.
- **Color Space Analysis**: computes statistical features from HSV, BGR, and LAB spaces to characterize the dominant color of each bounding box and to support more informed classification.
- **Robust Dominant Color Extraction**: applies K-means clustering to the bounding box pixels to estimate the dominant color in a way that is less sensitive to noise and mixed backgrounds.

These combined methods help ensure that color classification remains stable and reliable despite varying lighting and scene complexity.

2.1.2 Depth Estimation

For each valid and classified cone, its distance from the robot is estimated using the depth image received from the stereo camera. One significant challenge encountered during development was the inconsistent timing and reliability of the depth frames compared to the RGB stream. The stereo camera frequently produced invalid depth data or experienced lag relative to the RGB frames.

Synchronizing both camera streams was found to introduce excessive latency, which was not acceptable for a robot operating in motion. Instead, the system was designed to always use the most recent valid depth frame available, allowing for continuous operation without delays.

Another issue arose from the difference in resolution between the RGB preview and the full-resolution stereo depth image. The RGB camera originally captures images at 1280×720, but by default, it publishes a smaller square preview of size 250×250, which is a lower-resolution representation of the full image. This mismatch made it difficult to correlate the detected cone positions with accurate depth values.

To address this, we used `rqt` to reconfigure the RGB camera so that the published preview would be larger, specifically 400×400 pixels, and set to preserve the aspect ratio of the original image rather than that of the square preview. This produced a vertically stretched image, which we then resized in code to match the 1280×720 resolution of the depth image. Because the aspect ratio was preserved, this resizing did not introduce distortion and, in fact, corrected the stretched geometry. Increasing the preview size also reduced the loss of visual information compared to the default 250×250 output, although the final image remains slightly pixelated due to the limited resolution of the preview itself.

This resizing step was introduced not only to ensure consistency between RGB and depth frames, but also to improve cones detection, as the default preview configuration restricted the field of view so severely that nearby cones often fell outside the image.

To retrieve the depth value associated with a cone, the algorithm selects a point near the bottom-center of the bounding box, specifically, the center x-coordinate and the lower y-vertex, thus avoiding erroneous background readings. Because depth maps are often noisy or contain invalid values, especially in overexposed regions, the algorithm samples a small window around the selected pixel, ignores values of zero, and computes the median of the remaining depth values to obtain a stable estimate.

Detections that lack valid depth information after this step are discarded.

2.1.3 Publishing Results

For each processed frame, the final set of detected cones is published to a custom dedicated ROS topic named `/detected_cones`. To enable structured communication, two custom message types have been defined. The first, `ConeDetection`, represents an individual cone and contains the coordinates of the bounding box vertices, the position of the center point, the detected color, and the estimated distance from the robot. The second message, `ConeDetectionArray`, encapsulates a list of `ConeDetection` instances and is used to publish all valid detections corresponding to a single frame. This structure ensures that the information can be consumed efficiently by other modules in the system, such as those responsible for planning or control.

2.2 Pose Estimator Node

The `pose_estimator_node` is responsible for translating the cone detections into intermediate waypoints that guide the TurtleBot4 along a path that respects the navigation rules imposed by the colored cones. It subscribes to the custom topic `/detected_cones`, which publishes `ConeDetectionArray` messages containing the list of cones currently visible in the robot's field of view.

Once a message is received, the node selects the cone with the smallest valid depth value, identifying it as the closest cone in the scene. A filtering step is then applied: cones closer than 1.2 meters or farther than 3.5 meters are discarded. This range was determined empirically during testing, as both extremes tend to produce inaccurate waypoint estimates. When cones are too close, even small errors in pixel positioning or depth can lead to large deviations in the computed 3D point, while distant cones often result in waypoints that are misaligned with the actual scene or positioned far ahead of the robot's effective sensing range. In both cases, the resulting navigation behavior is suboptimal or unreliable, and thus these detections are excluded from further processing.

If the selected cone passes this filtering stage, the node proceeds to search for a second cone of the opposite color that, together with the first, can define a navigable passage. This search is limited to cones detected in the same frame, with a color different from that of the first cone, and whose bounding box center lies within a certain offset along the image's y-axis relative to the primary cone. Among all valid candidates, the one with the smallest depth is selected.

Once the pair is identified, the center points of their bounding boxes, originally expressed in image

coordinates, are projected into 3D camera coordinates. To perform this transformation, the node uses the intrinsic parameters published on the `/oakd/stereo/camera_info` topic. Although similar information is also available for the RGB camera, its parameters reflect the resolution of the preview rather than that of the full image. Since the detected cone coordinates refer to the resized 1280×720 RGB frame and the depth data comes from the stereo camera, only the stereo parameters ensure consistency between image resolution and depth measurements, making them the appropriate choice for accurate projection.

The two points are then transformed from the camera frame to the global `map` frame using the `tf2_ros` library, and specifically using the transformation between `oakd_rgb_camera_optical_frame`, shared by both RGB and stereo images, and `map`. The final waypoint is computed as the midpoint between the two cone positions in the map frame.

If no second cone is found, the node still attempts to generate a waypoint based on the single closest cone. The center of its bounding box is again projected into camera coordinates, and a lateral offset is applied along the x-axis of the camera frame to ensure the robot passes on the correct side of the cone. The sign of this offset depends on the cone's color and ensures consistent behavior with the navigation rules. The resulting point is then transformed into the `map` frame using the same transformation as before.

In both cases, whether using a cone pair or a single cone, the final waypoint is published as a `geometry_msgs/PoseStamped` message on the custom topic `/intermediate_waypoint`, making it available for the trajectory planner.

2.3 Navigation Node

The `navigation_node` is in charge of supervising the robot's movement from the starting position to the final goal, taking into account both the predefined objective and any intermediate waypoints generated in response to the constraints imposed by cones along the route. Its logic is implemented as a finite state machine composed of five states: `GO_FINAL`, `RECOVERY`, `NAVIGATING_MID`, `KIDNAP`, and `MISSION_COMPLETE`. These transitions are designed to ensure robust, reactive behaviour consistent with the dynamic nature of the task.

2.3.1 State Machine Overview

The system starts in the `GO_FINAL` state, since the robot's main objective is to reach the final destination. While in this state, the node instructs the **navigation stack** to begin planning toward the final goal and continuously monitors the navigation outcome. If the final goal is successfully reached, the state transitions to `MISSION_COMPLETE`, which marks the completion of the mission and handles resource cleanup before shutting down the node. If, instead, the navigation task fails, gets canceled, or is still running without termination, the system moves into the `RECOVERY` state to evaluate the current conditions and determine the appropriate next step.

Within `RECOVERY`, the node considers three possible transitions. The first involves the detection of a kidnapping scenario, such as the robot being lifted off the ground, by listening to the `/kidnap_status` topic, published by the **navigation stack**. If a kidnap event is detected, the state transitions to `KIDNAP`. To guarantee immediate reaction, the same transition is also available globally: whenever a kidnapping event is detected through the dedicated topic, the system can switch directly to `KIDNAP` regardless of the current state.

Alternatively, if the robot is still navigating but intermediate waypoints are being received, the node may enter the `NAVIGATING_MID` state. These waypoints are published on the `/intermediate_waypoint` topic by the `pose_estimator_node` and collected in a buffer, capped at 12 elements. If at least 4 waypoints are received, the robot assumes a detour is needed and prepares to navigate toward a computed intermediate pose.

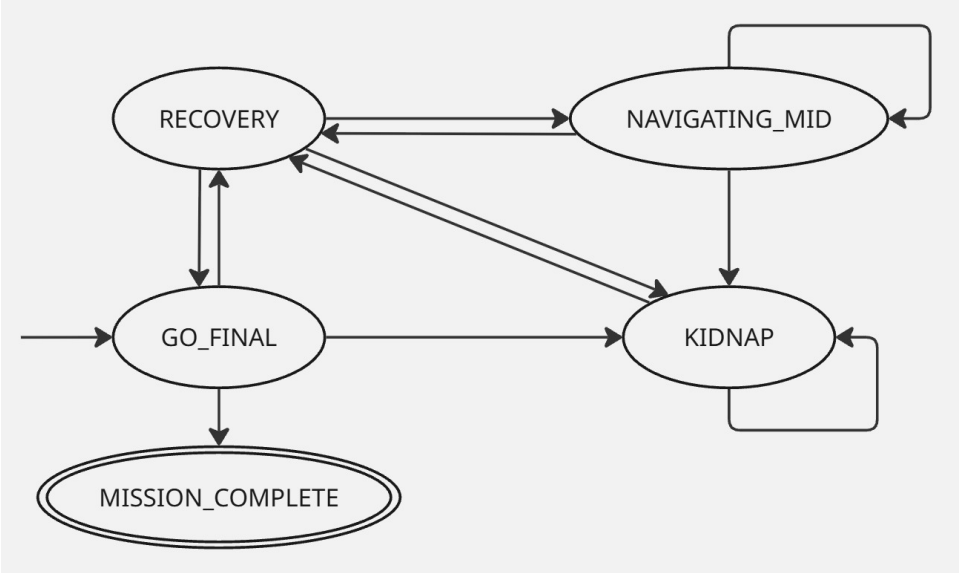

Figure 3: Design of the State Machine

If neither of the above conditions is met, i.e. no kidnapping is detected and no sufficient set of waypoints is received, the robot is assumed to still be on track, and the state returns to `GO_FINAL`.

The navigation node subscribes to the topic `/amcl_pose`, also published by the **navigation stack**, to track the robot's current estimated position, and it loads both the initial and final goal poses from a JSON configuration file at startup. The entire finite state machine is executed via a timer-triggered callback that fires every 0.5 seconds.

2.3.2 Kidnap Detection and Handling

The `KIDNAP` state is designed to manage situations in which the robot is physically lifted or repositioned in the environment, an event that can severely affect its localization and current navigation task. Its inclusion in the state machine is not only motivated by the need for robustness in real-world deployments, but also by one of the rules defined in the project specification, which explicitly allows repositioning the robot to help it detect cone gates it might have previously missed.

The robot detects a kidnapping event through the `/kidnap_status` topic, published by the **navigation stack**, which carries a boolean value indicating whether the robot is currently considered kidnapped. When this value becomes true, the state machine enters the `KIDNAP` state, and an internal flag is updated accordingly.

While the robot remains in this state and is still considered kidnapped, the system performs a cleanup routine: the current intermediate goal, the buffer of intermediate waypoints, and any active navigation task are cleared. This ensures that no residual or invalid navigation targets are pursued once the robot is placed back on the ground. The final goal is preserved, as it remains the long-term mission objective.

Once the robot is no longer considered kidnapped, a second cleanup is performed for safety. To support relocalization and improve the chances of detecting cone gates that might have been missed before the kidnapping event, the robot first rotates to align itself with the direction of the final goal. During this alignment, any new intermediate waypoints received are ignored and discarded to prevent inconsistent behavior due to rotational movement unrelated to cone detection.

After this alignment phase, the robot executes a sec-

ond in-place rotation of 60 degrees in the opposite direction. This additional movement is designed to expand the robot's field of view and allows it to capture any nearby cone pairs that were not previously visible. Unlike the first rotation, waypoints received during this second scan are retained and processed normally.

This strategy was chosen to strike a balance between necessary re-orientation and stability: allowing the robot to rotate freely after being repositioned would risk revisiting previously seen cones and generating outdated or redundant waypoints. Instead, by discarding detections during the alignment and only collecting new data during a controlled sweep in the opposite direction, the robot is able to resume navigation consistently and reliably. The only limitation of this approach is that if the robot is placed back into the environment facing a direction completely opposite to the intended path, the 60-degree scan may not be sufficient to recover missed cone gates. Nonetheless, this scenario is considered unlikely under the assumptions of reasonable repositioning.

At the end of the procedure, the state machine transitions back to `RECOVERY`, where any newly detected waypoints are processed as part of the usual intermediate navigation logic.

2.3.3 Intermediate Navigation Logic

While in the `NAVIGATING_MID` state, the node processes the buffered waypoints to estimate a stable and reliable intermediate pose to be sent to the **navigation stack**. This is done through a two-step filtering process designed to improve robustness against outliers. In the first step, the node computes the mean and standard deviation of all the poses in the buffer and retains only those located within a distance from the mean that is proportional to the standard deviation. Although this method helps exclude loosely scattered points, it may still be affected by the presence of significant outliers, which can distort the initial statistical estimates. To mitigate this, a second filter is applied using a fixed distance threshold relative to the previously computed mean, providing an additional safeguard against values that deviate excessively from the main distribution. The parameters for both filters were selected empirically based on repeated testing.

If the result of these filters is an empty set, no new goal is issued, and the robot retains its current navigation objective. Otherwise, the filtered waypoints are averaged, and the resulting pose is evaluated as a candidate goal. Before sending it to the **navigation stack**, it is compared with the currently active goal. If the two poses are spatially and directionally close enough, they are considered equivalent, and no update is issued. Otherwise, the new pose is submitted as the new intermediate goal.

If the robot is currently executing a navigation task, and it is found to be within a certain distance from the active intermediate goal, the buffer is cleared, the current task is explicitly canceled, and the state transitions back to `RECOVERY`. The reason for issuing a `cancelTask` rather than waiting for the current navigation task to finish is that, in ROS 2, there is often a notable delay between the termination of a task and the initiation of a new one. This delay is mainly due to the time required by the **navigation stack** to compute the new path. During this interval, when no goal is actively being tracked, the behavioral tree tends to enter a fallback routine that typically involves the robot spinning in place. This can result in the TurtleBot inadvertently detecting cones it has already passed, generating new, incorrect waypoints. By canceling the task slightly before completion and immediately issuing a new navigation request, this unintended behavior is avoided. In this condition, the TurtleBot either does not rotate at all or rotates only slightly, giving the **navigation stack** enough time to compute a new plan without falling into the fallback behavior.

3 Testing and Observations

3.1 Testing Setup

All tests were carried out in the second floor corridors of the Engineering building at the University of Salerno. This environment corresponds to the "known environment" mentioned in Section 1, for which a complete map was already available prior to the project. While the availability of the map enabled global planning, the physical corridors still introduced natural variability such as changing illumination and occasional dynamic obstacles.

The robot under test was the TurtleBot4, equipped with the sensing setup outlined in Section 1 and employed within the modules described in Section 2. In particular, RGB and stereo cameras supported cone detection and depth estimation, while the LIDAR was used for obstacle avoidance as part of the navigation stack. Hardware and environmental constraints that affected the testing process will be discussed later in Section 4.

Rather than following a formal benchmarking procedure, the evaluation was based on repeated observations of the robot's behaviour under realistic operating conditions. Cones were placed at different distances and in varying lighting conditions, including deliberately challenging cases such as highly reflective or overexposed areas. Parameter values (e.g., image filtering rules, valid depth ranges, state machine timing) were adjusted iteratively, and each change was validated by observing whether the robot's behaviour became more stable and consistent across multiple runs.

The following subsections summarise the main observations that guided the design of the cone detection, waypoint estimation, and navigation modules.

3.2 Cone Detection Observations

The evaluation of cone detection was carried out through repeated static and dynamic trials in the second floor corridors. Cones of both task colours were placed at different distances and orientations with respect to the TurtleBot4, and tests were repeated at various times of the day to capture illumination changes.

During the first experiments, the robot was manually repositioned and the RGB camera preview stream was used without resizing. We immediately noticed that the field of view was too narrow: for example, when the robot was standing at the entrance of a corridor, the camera could not frame both walls simultaneously, and cones close to the robot often fell outside the image. This observation led us to adopt the resizing strategy already presented in Section 2.1.3, which effectively widened the usable field of view and ensured that cones in relevant positions were captured.

Once this issue was addressed, attention turned to the behaviour of the detector itself. We observed that yellow cones were generally more difficult to detect than red ones, with a noticeably higher number of missed detections across different trials. More broadly, the overall reliability of cone detection was strongly dependent on lighting conditions. In particular, performance degraded significantly whenever a cone was directly illuminated by a strong light source or when such a bright area appeared in the background of the image. Under these conditions, detections were often lost altogether, or the bounding boxes produced by the network became unstable. At the same time, problems with colour recognition were also recurrent: cones were sometimes assigned to the wrong class, or even labelled as "unknown" and therefore discarded. These misclassifications were not random but appeared more frequently under variable or uneven lighting, for example when part of the cone was shaded while another part was brightly illuminated. Altogether, these observations motivated the introduction of the pre-processing pipeline described in Section 2.1.2, which improved both the stability of cone detections and the robustness of colour classification, even though extreme cases could still cause failures.

During these trials we also examined the depth images produced by the stereo camera, which we visualised

in `rqt`. We observed a high level of noise in the lighting conditions typical of the corridors and atrium: many pixels saturated either to zero or to very large distance values, making single-pixel estimates unreliable. These observations motivated the more robust strategy described in Section 2.1.3, where distances are computed from a small pixel window at the bottom-centre of the bounding box. In practice we compared different window sizes and tested both mean and median operators. The results showed that the median consistently provided more stable values in the presence of outliers, while the mean was easily distorted by saturated pixels. The adopted configuration was therefore selected empirically as the one that gave the most reliable estimates across repeated runs, with detections being discarded if even this approach returned inconsistent values.

Dynamic experiments, in which the robot was instructed to move between two points with cones positioned along the path, highlighted additional systematic sources of false positives. Fire extinguishers, doors, and wall signs frequently generated detections, and we observed that these objects consistently appeared in the **upper** region of the image. Floor stickers produced similar errors whenever they were partially visible, and in our tests this consistently occurred in the **lower** region of the frame. At the same time, we noticed that genuine cones falling in these peripheral regions were either too far away to be useful for navigation or so close that depth readings became unstable. These recurring observations directly motivated the positional filtering rules introduced in Section 2.1.1, which exclude narrow top and bottom bands of the frame to suppress irrelevant detections without discarding cones that matter for navigation. The exclusion masks were deliberately kept narrow to avoid interfering with valid detections, and their size was determined by testing different values during repeated trials.

Floor reflections introduced another recurring source of false positives. In these cases, we observed that the corresponding bounding boxes were always either unusually small or had distorted aspect ratios compared to real cones. This consistent pattern led us to implement the geometry-based filtering strategy in Section 2.1.1, which successfully eliminated these spurious detections in later tests.

Overall, the testing phase highlighted that cone detection was extremely sensitive to both camera configuration and environmental conditions.

3.3 Pose Estimator Observations

The pose estimator was evaluated exclusively through dynamic tests, where the robot was instructed to navigate between two points while cones were arranged in different configurations along the path. The objective was to assess both the accuracy and the reliability of the waypoints generated in motion, under varying cone placements.

During these experiments it became evident that depth measurements played a central role in the consistency of the waypoints. When cones were assigned very large depth values, often due to noise in the depth map, the resulting waypoints were placed far away from the cones themselves and were inconsistent with the actual layout of the corridor. This behaviour produced navigation targets that were fundamentally incorrect, undermining the ability of the robot to comply with the cone-passing rules. For this reason, an upper distance limit was introduced, beyond which detections were discarded.

A similar problem was observed when cones were detected at very close range. In this case, the waypoint tended to be placed well beyond the cones, at a distance that was greater than expected, and this delayed the computation of the subsequent waypoints. As a consequence, the robot frequently failed to respect the correct side of the following cones. To avoid these failures, a lower distance threshold was also introduced, below which detections were ignored. These limits, as described in Section 2.2, ensured that only cones within a

useful range contributed to waypoint generation.

Further tests analysed the behaviour when two cones were visible simultaneously versus when only one cone was detected. In the first case, the waypoint was computed as the midpoint between the two cones, and experiments confirmed that this solution reliably represented the correct passage. In the second case, the waypoint was generated from the position of the single cone with the addition of a lateral offset, whose direction depended on its colour. Testing focused on determining a value of this offset that would guarantee rule compliance without passing too close to either the visible cone or the corresponding cone of the opposite colour that was outside the field of view. Multiple trials were carried out with cones placed in straight sections as well as in narrow and wide curves, until a value was identified that consistently guided the robot to the correct side.

In summary, the range limits and the offset strategy were gradually introduced as direct responses to the issues repeatedly observed during these trials.

3.4 Navigation Observations

The navigation node was evaluated by repeatedly instructing the robot to move between predefined start and goal points while cones were arranged in different configurations along the path. These experiments focused on the stability of the state machine, the responsiveness of the navigation logic, and the reliability of the overall behaviour in conditions that included both expected transitions and unexpected events.

From the first trials it was clear that the execution frequency of the state machine had a strong influence on behaviour. With longer intervals the robot reacted too slowly to state changes and new waypoints, often missing the correct passage, while shorter intervals produced premature cancellations that disrupted ongoing tasks. Several values were tested until a period of 0.5 seconds emerged as the most suitable compromise, ensuring sufficient reactivity without overloading the system or interrupting valid navigation phases.

A second set of observations concerned the computation of intermediate goals. Forwarding poses directly from the pose estimator to the navigation stack quickly proved inadequate, since it produced oscillations, repeated replanning, and frequent misaligned trajectories. To address this, a buffer was introduced to accumulate waypoints before computing a new intermediate goal. Through repeated tests we observed that a minimum threshold of buffered detections was necessary to obtain stable results. When this threshold was set too high, the robot reacted too late, starting to steer towards the target only after missing the correct passage. When the threshold was too low, the response was faster but sometimes based on insufficient data. The adopted solution was therefore a small but non-trivial threshold of 4, which guaranteed timely reactions while still relying on consistent information.

Initially, the waypoint was calculated as the simple mean of the buffered detections. This already reduced instability compared to using raw inputs, but experiments revealed that the resulting goal could shift markedly away from the expected location. We then introduced a statistical filter, discarding poses that were too far from the mean relative to the standard deviation. This improved behaviour, but further testing showed that the mean and deviation themselves were still influenced by the outliers, and as a result some shifts persisted. To mitigate this, an additional fixed-distance threshold was introduced, preventing the waypoint from being displaced excessively from the cone pair that generated it. The actual value of this threshold was selected by testing several alternatives and observing which one ensured that updates occurred smoothly without moving the goal too far from the correct cone pair. For comparison, we also tested using only the fixed threshold, but the two-stage filter clearly outperformed the single approaches: using only the statistical filter left residual shifts, and using only the fixed threshold was less smooth in updating waypoints. The combined method therefore provided the best balance, ensuring that new goals were both responsive and faithful to the actual cone configuration.

As soon as the computation of intermediate goals was stabilised, another weakness became apparent at the moment when a goal was reached. Letting the navigation stack finalise a goal naturally introduced a delay before the next plan was issued, and in that window the robot frequently started to spin in place. This behaviour sometimes caused the system to pick up cones that had already been passed, which in turn produced invalid waypoints. The underlying cause is that, after completing a goal, the stack requires additional time to recompute the path toward the final destination and temporarily falls back to recovery routines. To address the issue we experimented with cancelling the goal once the robot was sufficiently close to it. This solution, already described in Section 2.3.3, consistently removed the unwanted rotations and resulted in smoother and more coherent transitions to the following waypoint.

Finally, specific experiments were dedicated to recovery after manual displacement. Without additional handling, the robot often attempted to follow outdated goals after being repositioned, leading to incoherent movements. The strategy described in Section 2.3.2 was therefore introduced, combining buffer resets, controlled rotations, and temporary suppression of detections during the alignment phase. In particular, the tests aimed to give the robot both the time and the means to re-localise itself and to detect cone pairs it had previously missed, without accidentally re-detecting cones already passed and thus moving backwards along the route. Several rotation amplitudes to the left and right were tested to determine how far the robot could scan without revisiting old pairs, under the assumption that the robot would not be repositioned in a direction inconsistent with the mission. Short pauses were also inserted between rotations, and their duration was tuned by conducting tests with different waiting times and selecting the value that allowed the robot to consistently recover its position before resuming navigation. With these measures in place, the robot resumed navigation more coherently after a kidnapping event, avoiding the inconsistencies observed in earlier trials.

By testing navigation in this way we were effectively testing the functioning of the entire system across different paths and conditions. These experiments allowed us to verify that the modules interacted as expected and that the chosen parameters for detection, pose estimation, and navigation were suitable to ensure coherent and rule-compliant behaviour. In addition, the variety of routes and cone configurations considered during navigation trials provided a system-level validation of the architecture: tests carried out on straight sections, narrow and wide curves, and differently illuminated areas confirmed both the correct interaction among the modules and the consistent enforcement of the colour-based passing rules. While some fragilities remained under extreme conditions, the set of solutions progressively introduced during testing proved sufficient to guarantee that the robot could complete different routes in a stable and coherent manner.

4 Limitations and External Constraints

4.1 Hardware Instability

One of the most significant obstacles encountered during the development phase was the hardware instability of the TurtleBot4 units. In particular, the first robot used for testing exhibited major issues with its onboard camera: the camera process would frequently crash or suffer from severe lag, and its parameters could not be modified. Attempts to reconfigure settings using `rqt`, either directly or via configuration files, were unsuccessful, with changes either being reset or triggering system failures. This was especially problematic because the camera was central to the task itself, serving as the primary source for both cone detection and depth estimation. Without a functioning camera, it was impossible to test or validate the core components of the system. These issues were largely mitigated when additional TurtleBots became available, allowing us to switch to more stable

units.

4.2 Battery and Availability Constraints

Another important limitation was related to power supply and hardware availability. All TurtleBots required long charging times but discharged quickly, especially when using power-intensive sensors such as the LIDAR. This significantly reduced the effective time available for real-world testing. The situation improved only when we gained access to more robots and were able to rotate between them, allowing one robot to charge while another was in use.

4.3 Testing Environment and Device Limitations

Further challenges were imposed by the physical testing environment. The corridors where the experiments were conducted did not have accessible power outlets. Even if they had been present, they would have been of limited use: during test runs, it was necessary to follow the robot closely to prevent loss of communication between the laptop and the TurtleBot, making stationary charging impractical. Additionally, only one of the three laptops available to the team could be used for testing. The other two machines, when not connected to a power source, were unable to run even the lightest custom ROS 2 nodes without severe lag, freezing, or spontaneous shutdown. This behavior has never occured under Windows before, leading us to suspect a compatibility issue with Ubuntu or with how power management was handled by those specific machines.

References

[1] EngrAwab. Safety cone detection using yolov8. `https://github.com/EngrAwab/Safety_Cone_detection`, 2023.

[2] Ziyu Lin, Yunfan Wu, Yuhang Ma, Junzhou Chen, Ronghui Zhang, Jiaming Wu, Guodong Yin, and Liang Lin. Yolo-llts: Real-time low-light traffic sign detection via prior-guided enhancement and multi-branch feature interaction. 2025.